Phase 4 | Project Planning Phase

Project Planning

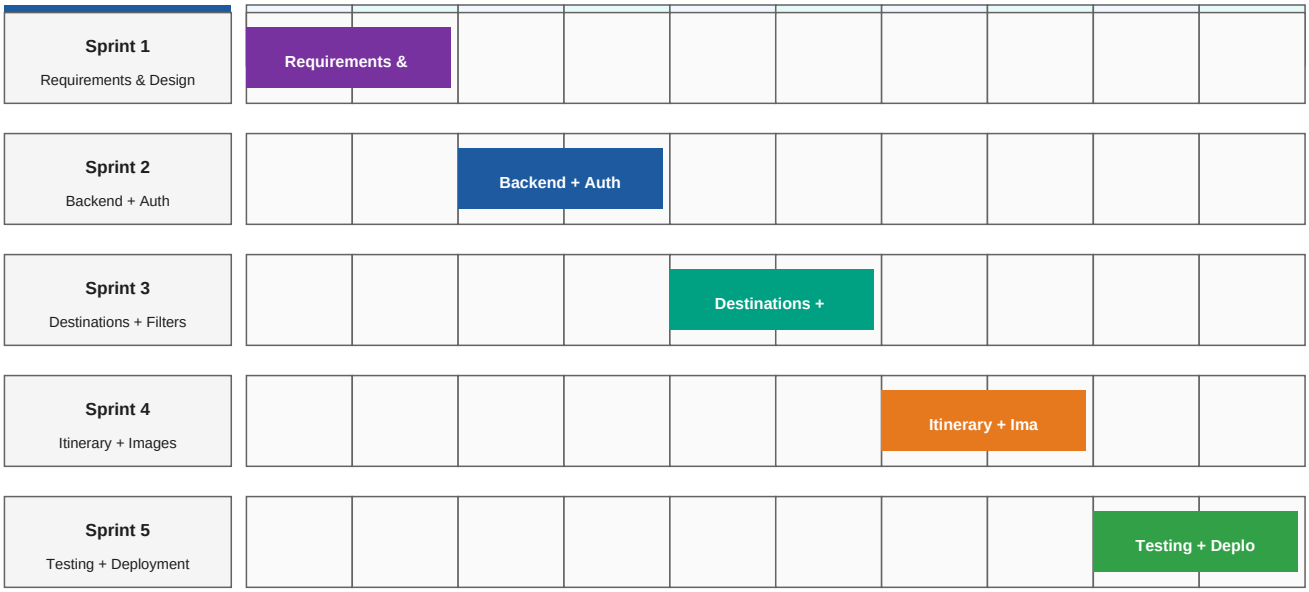
AI-Powered Travel Planner | Smart Situational Itinerary Engine

Live URL:	https://travel-planner-iota-plum.vercel.
GitHub:	github.com/itskian2007/travel-planner
Tech:	React + FastAPI + PostgreSQL + Vercel
Team:	itskian2007

1. Project Scope

The Travel Planner project spans 10 development weeks, organized into 5 phases: Requirements, Design, Development, Testing, and Deployment. The project follows an Agile-Scrum methodology with 2-week sprints and daily standups.

2. Sprint Timeline / Gantt Chart



3. Sprint Breakdown

Sprint 1 (Wk 1-2)

- User research & problem statement definition
- Empathy mapping & persona creation
- Technology selection & justification
- System architecture design
- Database schema design
- API endpoint specification

Sprint 2 (Wk 3-4)

- FastAPI project setup & configuration
- PostgreSQL database setup & migrations
- User registration endpoint (POST /register)
- User login endpoint with JWT (POST /login)
- Forgot password flow
- React project setup with TailwindCSS

- All 20 destination data structures built

- Filter engine: Mood / Budget / Diet / Persona

- Accessibility preference module

- Destination browsing page with cards

- Filter panel UI with real-time updates

- Wikipedia API image integration

Sprint 4 (Wk 7-8)

- Day 1-3 curated itinerary data for all 20 destinations
- Procedural Day 4-10 engine (contextual generation)
- Itinerary display page with day cards
- 304 unique image verification & deduplication
- Full itinerary API endpoint
- Landmark image rendering with fallbacks

Sprint 5 (Wk 9-10)

- Unit testing: auth, filter, itinerary modules
- Integration testing: frontend <-> backend API
- Performance testing: load & response time
- Security audit: JWT, SQL injection, XSS
- Vercel deployment configuration
- Final documentation & README

4. Risk Register

ID	Risk Description	Likelihood	Impact	Mitigation Strategy
R-01	Wikipedia API Rate Limiting	HIGH	HIGH	Cache images after first fetch, fallback images
R-02	PostgreSQL Connection Timeout	MEDIUM	HIGH	Connection pooling, retry logic
R-03	Vercel Build Failures	LOW	HIGH	Local build checks, GitHub Actions CI
R-04	Duplicate Image Detection	MEDIUM	HIGH	Hash-based deduplication before storage
R-05	JWT Token Theft	MEDIUM	HIGH	HTTPS only, short expiry, refresh tokens
R-06	Slow Itinerary Generation	LOW	MEDIUM	Async endpoints, pre-computed templates
R-07	Scope Creep	MEDIUM	MEDIUM	Strict MoSCoW prioritization per sprint